

D2587R1

to_string or not to_string

Draft Proposal, 2022-06-03

Author:

[Victor Zverovich](#)

Audience:

LEWG

Project:

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Table of Contents

1 Introduction

2 Revision history

3 Examples

4 Proposal

5 Impact on existing code

6 Implementation

7 Wording

References

Informative References

"Though this be madness, yet there is method in 't." — Polonius

§ 1. Introduction

C++11 introduced a set of `std::to_string` overloads for integral and floating-point types. Fortunately for integral and unfortunately for floating-point overloads they are all defined in terms of `sprintf` inconsistently with C++ formatted output functions ([N4910]). Additionally, the choice of the floating-point format makes `std::to_string` of very limited use in practice. This paper proposes fixing these issues while retaining existing semantics of integral overloads.

§ 2. Revision history

Changes since R0:

- Added an entry to Annex C.
- Added a feature test macro.
- Applied the same changes to `to_wstring` in the wording.

§ 3. Examples

Consider the following example:

```
auto loc = std::locale("uk_UA.UTF-8");
std::locale::global(loc);
std::cout.imbue(loc);
setlocale(LC_ALL, "C");

std::cout << "iostreams:\n";
std::cout << 1234 << "\n";
std::cout << 1234.5 << "\n";

std::cout << "\nto_string:\n";
std::cout << std::to_string(1234) << "\n";
std::cout << std::to_string(1234.5) << "\n";

setlocale(LC_ALL, "uk_UA.UTF-8");

std::cout << "\nto_string (uk_UA.UTF-8 C locale):\n";
std::cout << std::to_string(1234) << "\n";
std::cout << std::to_string(1234.5) << "\n";
```

It prints:

```
iostreams:
1 234
1 234,5

to_string:
1234
1234.500000

to_string (uk-UA.UTF-8 C locale):
1234
1234,500000
```

Since `std::to_string` uses the global C locale and no grouping the integral overloads are effectively unlocalized. The output of floating-point overloads is inconsistent with that of `iostreams` because the former takes the decimal point from the global C locale and doesn't do grouping.

Additionally, due to an unfortunate choice of the fixed format in the floating-point overloads they are only useful for numbers in a limited exponent range. For example:

```
std::cout << std::to_string(std::numeric_limits<double>::max());
```

prints

```
179769313486231570814527423731704356798070567525844996598917476803157260786
387605895586327668781715404589535143824642343213268894641827684675467035375
604991057655128207624549009038932894407586850845513394230458323690322294816
559332123348274797826204144723168738177180919299881250404026184124858368.00
```

(line breaks inserted for readability)

Here only the first 17 digits are meaningful, the next 292 are so-called "garbage" digits ([\[DRAGON\]](#)). And finally we have 6 meaningless zeros after a possibly localized decimal point.

Formatting of small floating-point numbers is even less useful. For example:

```
std::cout << std::to_string(-1e-7);
```

prints

```
-0.000000
```

In fact almost half of floating-point numbers are formatted as zero and there is a precision loss or garbage digit output in many other cases.

Unfortunately issues with floating-point overloads of `std::to_string` have not been discussed in the paper that proposed this facility ([N1803], [N2408]). However, they were brought up as early as 2015 in a mailing list discussion ([STD_DISCUSSION]) which proposed deprecating floating-point overloads of `std::to_string` but no paper came out of that.

§ 4. Proposal

Redefine `std::to_string` in terms of `std::format` which in turn uses `std::to_chars` making more explicit the fact that integral overloads are unlocalized and changing the format of floating-point overloads to also be unlocalized and use the shortest decimal representation.

The following table shows the changes in output for the following code:

```
setlocale(LC_ALL, "C");  
auto output = std::to_string(input);
```

input	output	
	before	after
42	42	42
0.42	0.420000	0.42
-1e-7	-0.000000	-1e-7
1.7976931348623157e+308	1797693134862315708145274 2373170435679807056752584 4996598917476803157260780 0285387605895586327668781 7154045895351438246423432 1326889464182768467546703 5375169860499105765512820 7624549009038932894407586 8508455133942304583236903 2229481658085593321233482	1.7976931348623157e+308

input	output	
	before	after
	7479782620414472316873817	
	7180919299881250404026184	
	124858368.000000	

and similarly with the global C locale set:

```
setlocale(LC_ALL, "uk_UA.UTF-8");
auto output = std::to_string(input);
```

input	output	
	before	after
12345	12345	12345
1234.5	1234,500000	1234.5

§ 5. Impact on existing code

This change will affect the output of `std::to_string` with floating-point arguments. In most cases it will result in a more precise and/or shorter output. In cases where the C locale is explicitly set the decimal point will no longer be localized.

§ 6. Implementation

{fmt} implements proposed changes in `fmt::to_string`.

§ 7. Wording

Add an entry for `__cpp_lib_to_string` to section "Header `<version>` synopsis [[version.syn](#)]", in a place that respects the table's current alphabetic order:

```
#define __cpp_lib_to_string **placeholder** // also in <string>
```

Modify subsection "Numeric conversions [\[string.conversions\]](#)":

```
string to_string(int val);
string to_string(unsigned val);
string to_string(long val);
string to_string(unsigned long val);
string to_string(long long val);
string to_string(unsigned long long val);
string to_string(float val);
string to_string(double val);
string to_string(long double val);
```

~~7 Returns: Each function returns a string object holding the character representation of the value of its argument that would be generated by calling `sprintf(buf, fmt, val)` with a format specifier of `"%d"`, `"%u"`, `"%ld"`, `"%lu"`, `"%lld"`, `"%llu"`, `"%f"`, `"%f"`, or `"%Lf"`, respectively, where `buf` designates an internal character buffer of sufficient size.~~

7 Returns: `format("{}"_s, val)`.

```
wstring to_wstring(int val);
wstring to_wstring(unsigned val);
wstring to_wstring(long val);
wstring to_wstring(unsigned long val);
wstring to_wstring(long long val);
wstring to_wstring(unsigned long long val);
wstring to_wstring(float val);
wstring to_wstring(double val);
wstring to_wstring(long double val);
```

~~14 Returns: Each function returns a `wstring` object holding the character representation of the value of its argument that would be generated by calling `swprintf(buf, buffsz, fmt, val)` with a format specifier of `L"%d"`, `L"%u"`, `L"%ld"`, `L"%lu"`, `L"%lld"`, `L"%llu"`, `L"%f"`, `L"%f"`, or `L"%Lf"`, respectively, where `buf` designates an internal character buffer of sufficient size `buffersz`.~~

14 Returns: `format(L"{}"_s, val)`.

Add to Annex C (informative) Compatibility [\[diff\]](#) the following new subclause:

C.? C++ and ISO C++ 2026 [\[diff.cpp23\]](#)

This subclause lists the differences between C++ and ISO C++ 2023 (ISO/IEC 14882:2023, Programming Languages — C++), by the chapters of this document.

C.?.1 Clause 23: strings library [diff.cpp23.strings]

Affected subclauses: 23.4

Change: Output of floating-point overloads of `to_string` and `to_wstring`.

Rationale: Prevent loss of information and improve consistency with other formatting facilities.

Effect on original feature: `to_string` and `to_wstring` function calls that take floating-point arguments may produce a different output. For example:

```
auto s = std::to_string(1e-7); // "1e-7"  
// previously "0.000000" with '.' possibly  
// changed according to the global C locale
```

§ References

§ Informative References

[DRAGON]

Guy L. Steele Jr.; Jon L White. [How to Print Floating-Point Numbers Accurately](https://fmt.dev/papers/p372-steele.pdf). URL: <https://fmt.dev/papers/p372-steele.pdf>

[N1803]

Pete Becker. [Simple Numeric Access](https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2005/n1803.html). URL: <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2005/n1803.html>

[N2408]

Pete Becker. [Simple Numeric Access Revision 2](https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2007/n2408.html). URL: <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2007/n2408.html>

[N4910]

Thomas Köppe. [Working Draft, Standard for Programming Language C++](https://wg21.link/n4910). URL: <https://wg21.link/n4910>

[STD_DISCUSSION]

[Deprecate to_string\(\) for floating point types?](https://groups.google.com/a/isocpp.org/g/std-discussion/c/6xhWgsEuvPo). URL: <https://groups.google.com/a/isocpp.org/g/std-discussion/c/6xhWgsEuvPo>